Action-Time versus Review-Time Authorization Status across the EU Trusted Lists: The Time-Travel Evidence Lab

Anton Sokolov — Tyche Institute, Tallinn, Estonia

2026-06-29

Corresponding author: Anton Sokolov — Tyche Institute, Mustamäe tee 167-50, Tallinn 12913, Estonia — anton.sokolov@tyche.institute — ORCID 0000-0003-2452-7096.

Highlights

- On 26 EU trusted lists, 2,071 of 4,815 entries flip valid-to-revoked.
- Same action reads withdrawn under a current review but granted on dated replay.
- Action-time and review-time authorization status are two dated facts, not one.
- Lose the dated context and the later accountability verdict is unreproducible.
- Standards secure the key/certificate layer, not the delegated-mandate layer.

Abstract

A delegated action — a signature, or a software agent acting for someone — is often reviewed long after the fact, once its permissions have been revoked, trust services have changed state, and records have moved. The security problem is that the mechanisms used to re-check it — status lists, token introspection, trusted lists, timestamps, logs — answer "is this valid?" as a single present-time query, confusing two dated facts: the status at action time and at review time. That conflation makes a later accountability verdict non-reproducible: the same action is judged differently on different days for reasons external to it. We demonstrate the effect on the real published EU member-state Trusted Lists: across 26 of the 27 lists, 2,071 of 4,815 trust-service entries (43%, every list affected) record a valid-to-revoked transition, so an action performed while a qualified service was valid reads as withdrawn under a current-only review yet valid under dated replay. Two controlled harnesses isolate the mechanism one layer up — the delegated mandate that no published list records — and show that a later reviewer must keep the dated dependency closure around the record. We position the result against long-term-validation, evidence-record, and secure-logging standards, which preserve validity, existence, or log integrity over time but model revocation at the key and certificate layer, not the mandate layer

that review turns on. The contribution is a claim-conservative, reproducible account of which interface semantics must be preserved, and of the residual trust it relocates rather than removes.

Keywords: evidence preservation; long-term validation; revocation; trusted lists; authorization status; delegated authority; non-repudiation; digital forensics.

1. Introduction

When someone acts with permission — a person signing a document, or a software agent acting on their behalf — the proof that the action was allowed is plain at the time: the permission is active, the verifier knows which policy applies, the issuer is in good standing, and the status records are fresh. Accountability, however, is exercised later, and by then that clarity is gone. A reviewer reconstructs the action after the permission has been revoked, a trusted-list entry has changed, a status endpoint has rotated or gone offline, and the assumptions that made the original evidence legible are no longer present. Autonomous software agents acting for a principal under delegated authority are the pressing contemporary instance, since they act at machine speed and at volume, but the problem is the general one of reviewing any delegated action once its authorisation context has moved.

A credential status list encodes the present revocation bit [1]. OAuth token introspection returns the active or inactive state at the instant of the call [2], as does its successor work on grant negotiation [3]. A trusted list publishes the set of qualified providers as it stands now [4, 5]. Each answers “is this valid?” as a single, current-time query. That is exactly what a relying party deciding in the moment needs, and these mechanisms do it well. The difficulty appears only when the question is moved in time: a reviewer who re-runs the same query receives the current answer and cannot, from it alone, separate “revoked at review” from “valid when the action was performed”.

Practice and standards treat authorisation status as a single time-indexed fact, retrieved on demand. For later review it is two dated facts: the status that obtained at action time, and the status that obtains at review time. When the two are collapsed into one live lookup, the later verdict inherits whatever the world looks like at the moment of review, and the same action can be judged differently on different days for reasons that have nothing to do with the action. An accountability verdict that depends on when it is asked is not reproducible.

This sets the questions the article answers. The overarching question is whether, once authorisation status has drifted between the time a delegated action is performed and the time it is later reviewed, a reviewer can reproduce the action-time accountability verdict, and what an evidence interface must preserve and replay for that verdict to be reproducible by a party other than the original relying party. It is settled through four sub-questions, each answered by one part of the study:

- **RQ1** (Section 5.1): on the real, externally published trust artefacts of the EU, does a current-only status lookup diverge from a dated point-in-time replay for the same fixed action, and how widespread is the divergence-enabling transition

across the member-state Trusted Lists?

- **RQ2** (Section 5.2): does the same divergence arise one layer up, at the delegated mandate and its scope that no published list records, when a revocation is timed and a status responder later disappears?
- **RQ3** (Section 5.3): is preserving the signed evidence record sufficient for a reproducible bounded verdict, or is the unit of preservation the dated dependency closure around it?
- **RQ4** (Section 6): what minimal signed evidence object and verification predicate satisfy that preservation requirement, what security guarantees do they give against which adversary capabilities, and what residual trust do they relocate rather than remove?

In plain terms, we design a signed *mandate-status statement* and a predicate that decides whether an action-time authorisation verdict can be replayed, and we show across 26 EU member-state trusted lists that the same action can read `withdrawn` under a current-only review yet valid under dated replay, with 2,071 of 4,815 service entries (43%, every list affected) carrying the valid-to-revoked transition that produces the divergence. We study these questions with the Time-Travel Evidence Lab, a reproducible case study that holds one fixed delegated action stable and changes only the review context around it. Holding the action fixed means that when a verdict changes, the explanation must point to changed evidence context, not to a different action. The effect is shown first on a real, externally published trust artefact and then isolated in two controlled harnesses. We are explicit about what rests on what: the trusted-service layer is grounded on real published data, while the delegation and dependency results are synthetic illustrations of the mandate layer that no published list yet records, so the natural next validation is a real-system instantiation of the mandate-status statement over a live delegation protocol with an executed revocation. The study is claim-conservative and reproducible from a deposited package, and makes no claim about real-world incident rates beyond what the published lists record, nor of production performance, legal effect, standards conformance, or absolute novelty.

The contributions answer these questions in turn.

First, we measure the conflation on the real published EU member-state Trusted Lists (RQ1, RQ2). An action performed while a qualified service was valid reads as `withdrawn` under a current-only review today yet valid under dated replay, and 2,071 of 4,815 service entries across 26 of the 27 member-state lists carry such a transition in their own history — every list affected, 45 of 66 on the Estonian list — so the divergence is a property of real published data across the EU trust infrastructure rather than a constructed case. A controlled delegation harness then reproduces the same split one layer up: current-only lookup returns `valid_currently`, then `revoked_at_review`, then `stale_or_missing_status` as the world moves on, while dated replay returns `valid_when_performed` throughout, at the mandate and delegation layer that no published list records.

Second, we show that the unit of preservation is the dependency closure around the record, not the record alone (RQ3). A sixteen-scenario dependency matrix demonstrates that keeping the primary evidence object while losing its delegation status, schema, issuer metadata, or explanation trace leaves the bounded verdict

unreproducible. What a later reviewer needs to keep is the dated bundle in which the signed file is only the centre.

Third, we design the evidence object that meets the gap and locate it against the security mechanisms that already exist for verifying things later, and we are explicit about the residual trust our own remedy carries (RQ4). Long-term-validation, evidence-record, and tamper-evident-logging schemes preserve cryptographic validity, the existence and integrity of data, or the integrity and ordering of log entries across time, but they reason about the key, certificate, and entry layer, not the mandate and delegation layer; authenticated-delegation schemes scope and authenticate authority at issue time but do not preserve a dated snapshot of it for replay; and dated replay relocates trust to the party who curated and signed the archive rather than removing it. We treat that residual as a named threat-model item, bounded by the field's own renewal, preservation, and transparency mechanisms.

The remainder is organised as an artefact-first design-and-evaluation study, in which the real-list measurement is the empirical anchor and the mandate-status statement the designed contribution. Section 2 sets out the threat model and trust assumptions; Section 3 is the related work, positioning the article against neighbouring standards and naming what each leaves unsolved; Section 4 states the research design and method; Section 5 is the demonstration and experiments; Section 6 presents the designed mandate-status statement, its verification predicate, and the security argument; Section 7 states the limitations including the archive-trust residual; and Section 8 concludes by answering the research questions.

2. Threat model and trust assumptions

The setting is one synthetic delegated action performed at action time T and reviewed for accountability at review time $T+k$, after status has drifted. Three actors structure the problem, and the question each can and cannot answer organises the rest of the paper. These unanswerable questions are what RQ1–RQ4 make precise, and the security goals of Table 6 are designed to meet.

The producer (at T). The producer is the system that performs the delegated action and emits its record. Its design choice is whether to capture a dated action-time authorisation snapshot — the delegation grant and its scope, the issuer's trusted-list status, and the dependency closure — or to emit only the action and rely on a later party re-deriving status by live lookup. An adversarial or merely careless producer emits the bare action. The assurance question is whether a record produced at T lets a future reviewer establish valid-when-performed without re-contacting status responders that may by then be offline or defunct. A current-only interface cannot supply this, because nothing in it is dated to T.

The archive curator (between T and $T + k$). If an action-time snapshot is preserved, some party curates and signs it. This actor controls what was captured, whether it was complete and honest at capture, and the key that binds it. Here the model concedes its central limitation rather than hiding it: archived action-time replay does not eliminate trust, it relocates it to the curator. A curator who signs a faithfully preserved but wrong snapshot yields a verifiable wrong record; the scheme

guarantees the snapshot is unchanged since capture, not honest at capture. We return to this residual in Section 7 and bound it there, because the curator is not an arbitrary party: it is a named, timestamp-anchored signer whose misbehaviour is made detectable, accountable, or renewable by existing mechanisms.

The later verifier (at $T + k$). The reviewer chooses which trust anchors to accept at verification time and which dated artefact to replay. Its adversary controls the availability of original responders and dependency endpoints, and any incentive to let the reviewer default to a current-only lookup that is biased toward `revoked_at_review`. All verification bottoms out in at least one accepted anchor [6]; the residual the verifier must accept — that the timestamping anchor and the signature and hash algorithms are unbroken at verification time — is stated explicitly rather than assumed away. The contrast the paper measures is between this verifier replaying a dated archive, which is reproducible, and the same verifier performing a current-only lookup, which is not.

The verdict labels used throughout are local vocabulary entries emitted by a bounded synthetic verifier, not legal, certification, or production outcomes. They describe what the verifier can conclude under the supplied, dated assumptions, and each positive result is paired with a negative control. # 3. Related work: what the standards leave unsolved

Long-term validation and evidence-record standards are the backbone of "verify it later". The paper credits each, and for each it names the specific thing left open for recovering action-time authorisation context after revocation under temporal review drift. The pattern across all of them is the same: they make either cryptographic validity or the existence and integrity of a datum survive across time, and they model revocation at the key and certificate layer. None preserves and replays the delegated mandate and its status as a dated fact travelling with the action.

Current-status interfaces. Credential status lists and OAuth token introspection answer "is this valid?" as a single current-time query: introspection returns the active or inactive state at the moment of the call [2, 3], and a status list encodes the present revocation bit [1, 7]. They were built for a relying party deciding now and do that well. What they leave unsolved is the two-dated-facts problem of our delegation-revocation experiment: neither preserves a dated, replayable snapshot of the grant and its scope as it stood at T, so a reviewer at $T + k$ who re-queries receives only the current bit and cannot separate revoked-at-review from valid-when-performed.

Trusted lists. EU trusted lists publish the current set of qualified providers and carry dated service-status histories, so some reconstruction is in principle possible [4, 5]. Section 5.1 turns exactly this property into the experiment: the real Estonian list records, for each service, the dated transitions a point-in-time replay needs, yet a relying party is expected to consult the list as published now, and a current-only read of a service withdrawn since the action returns the present status with no bound snapshot of whether the issuer was eligible, and under what status, when the action was performed. The history exists; what is missing is action-time membership as a first-class, signed, portable artefact that travels with the action and is replayed by default rather than reconstructed out of band.

Existence and transparency. RFC 3161 time-stamping binds a hash to a time but

is deliberately content-, identity-, and validity-neutral [8, 9]; it descends from the hash-linking time-stamping of Haber and Stornetta [10] and is realised at scale by keyless, server-side time-stamping that reduces reliance on any one signing key for the time anchor [11]. Certificate Transparency logs that a certificate was issued or observed, and by its own design records existence, not validity, deferring revocation status to other mechanisms [12, 13]. Against our dependency-closure experiment, these fix and make auditable individual data, but do not preserve the transitive set of authorisation dependencies — delegation chain, issuer trusted-list state, sub-grant scope — that must be dated and replayed together for an action-time verdict to be reproducible.

Registered statements. SCITT and Sigstore register signed statements about artefacts into transparency services and return inclusion receipts [14, 15], while C2PA binds signed provenance manifests to content rather than returning transparency-log receipts by default [16]. SCITT is the closest neighbour, targeting accountable registered actions. Its gap is the reconstruction of action-time authorisation after later revocation: a receipt proves a statement was registered under some policy by an issuer, but registration proves production by an issuer, not the truth of the statement, and policy beyond signature checks is implementation-defined. It standardises verifiable transparency of the act without a reconstructable record of the action-time delegation and trusted-list context.

Long-term signature validation. AdES long-term validation and the underlying validation procedures give the strongest existing retrospective-validity model: proof of existence, best-signature-time, and long-term-validation augmentation let a verifier establish, after the fact, that a signing certificate was not revoked or expired when it signed [17]. Mapped to delegation-revocation drift, its gap is the layer above the key. The model reasons over certificate-path validity and revocation at best-signature-time, and stops short of whether the delegated mandate or scope was in force and unrevoked at T. Its retrospective validity stops one layer below the authorisation layer that delegated-authority accountability turns on.

Evidence records. The IETF evidence-record syntaxes provide long-term non-repudiation of existence and integrity through Merkle hash trees and renewal chains [18, 19]. By their own scope they prove existence and integrity, explicitly not validity. They are a strong container for an action-time authorisation-context record but assert nothing about that record's meaning or validity when the action occurred; they would faithfully preserve a delegation snapshot without establishing it was in force at T.

Governance frameworks. AI risk-management frameworks frame accountability and traceability as governance outcomes but are process-level and technology-neutral [20]: they call for documentation and traceability without specifying a verifiable, dated authorisation-context record. What they leave unsolved across all three experiments is the concrete artefact and verification predicate that separates authorised-when-performed from revoked-at-review.

Secure and forward-secure logging. A substantial security literature protects audit logs against tampering by an adversary who controls the logging host. Schneier and Kelsey chain evolving one-way keys so that entries written before a

compromise cannot be read or altered undetectably [21], building on the forward-integrity property of Bellare and Yee, whose forward-secure authentication keeps pre-exposure entries verifiable after key capture [22], while Crosby and Wallach give tree-based structures whose tamper-evident membership and consistency proofs are logarithmic rather than linear in size, holding an untrusted logger honest [23], a line continued by forward-secure aggregate schemes for host-compromise resilience [24]. Their security objective is the integrity and append-only ordering of log entries as bit-strings. They do not reconstruct what an entry asserts: the record "delegate D was authorised under mandate M" may be perfectly intact and correctly ordered while the authorisation it describes was revoked, narrowed, or allowed to lapse between issuance and review. The threat here is status drift, not entry tampering, so a verifier replaying a tamper-evident log still cannot recover the dated mandate state that held when the delegate acted.

Authenticated delegation for software agents. A recent line establishes how authority is delegated to software agents and authenticated at the moment an agent acts. South et al. extend OAuth 2.0 and OpenID Connect with agent-specific delegation credentials that bind a principal to an agent under scoped, signed permissions [25], over the token-exchange primitive of RFC 8693, whose act and sub claims record that one party acts on behalf of another at the point a token is issued [26]. Practitioner guidance converges on the same issue-time framing: the OWASP agentic threat taxonomy treats excessive or stale agent permissions as a deployment-time hazard [27], and the Cloud Security Alliance advocates short-lived, context-aware agent identities issued for the current task [28]. Each answers who may act, with what scope, authenticated now. None preserves a dated, replayable snapshot of the mandate as it stood when the action was taken, so once the credential is revoked or lapses a later reviewer cannot reproduce the action-time verdict, which is the gap the delegation experiment of Section 5.2 isolates.

The synthesis is that revocation is everywhere modelled at the credential layer (was the certificate revoked) rather than the mandate layer (was the delegated authority revoked, narrowed, or never conferred at T). The secure-logging and authenticated-delegation lines are the closest security neighbours: the first preserves the entries, the second authenticates the grant, and neither replays the dated mandate status a later verdict turns on. Demonstrated on one fixed action across a real published trusted list, a delegation-revocation harness, and dependency closure, that missing layer is what the paper isolates. Two companion notes supply the governance and trust-infrastructure vocabulary the lab borrows without re-deriving [29, 30]. These gaps are the requirements the designed artefact of Section 6 must meet; Section 4.4 carries them forward as explicit solution objectives.

Table 1 summarises the section: for each neighbouring family it names the layer the family reasons at, what it makes survive over time, what it leaves open for reconstructing action-time authorisation status, and the experiment that exercises that gap. The references for each family are those introduced in the prose above.

Table 1: Where each neighbouring standard family reasons, what it preserves over time, what it leaves open for reconstructing action-time authorisation status, and the experiment that exercises the gap. The status-drift and dependency-closure gaps are what the Time-Travel Evidence Lab measures.

Standard family	Reasons at	Preserves over time	Leaves open for action-time review	Tested in
Current-status interfaces	present validity bit	current grant or revocation state at the call	a dated, replayable snapshot of the grant and scope at T	§5.2
Trusted lists	certificate, trust service	current provider set, with a dated status history	action-time membership as a signed, portable artefact replayed by default	§5.1
Existence and transparency	data existence and time	existence and a time anchor, validity-neutral	the transitive dependency set, dated and replayed together	§5.3
Registered statements	a registered statement	that a statement was registered by an issuer	the action-time authorisation behind the statement after revocation	§5.2
Long-term signature validation	certificate-path validity	a signing certificate valid at best-signature-time	whether the mandate or scope was in force at T	§5.2
Evidence records	bytes: existence, integrity	existence and integrity across renewal, not validity	the meaning of the preserved record at T	§5.3
Governance frameworks	process and governance	documentation and traceability as outcomes	a concrete dated authorisation-context artefact and predicate	§5.1–5.3
Secure, forward-secure logging	log-entry integrity, order	tamper-evidence and append-only entry ordering	the dated mandate state an entry asserts	§5.2
Authenticated delegation	an issue-time grant	who may act, with what scope, authenticated now	a dated, replayable snapshot of the mandate at T	§5.2

4. Research design

This is an artefact-first design-and-evaluation study in the constructive tradition common to systems-security research: we characterise a security problem, measure it on real data, design a mechanism to remedy it, and evaluate that mechanism against an explicit threat model with negative controls. The research output is a

designed evidence object and its verification predicate (Section 6), motivated and demonstrated by a measurement on a real published trust artefact (Section 5.1) and exercised by two controlled harnesses (Sections 5.2 and 5.3). The work follows the security field's native threat-model, design, and evaluation form of design-oriented research rather than importing an information-systems design-science apparatus; readers who prefer that lineage may read Section 2 as problem identification, this section and Section 6 as the statement of solution objectives and the design, and Sections 5 and 6 together as demonstration and analytical evaluation.

4.1 Approach, paradigm, and unit of analysis

The unit of analysis is the verifier verdict over one fixed delegated-action evidence episode under a varied review context. The action is held fixed by design so that any change in verdict is attributable to changed evidence context, not to a changed event. The study seeks internal consistency and reproducibility of a mechanism, not statistical generalisation or an external truth oracle: the verdict labels are local vocabulary emitted by a bounded synthetic verifier, the harnesses are author-built, and the evaluation establishes that the verifier returns the specified verdict under the specified dated context, with a falsifying negative control for each positive result. This is a deliberate scope choice, not an unstated limitation; its consequences for external validity are taken up in Section 7.

4.2 Case and data selection

The empirical anchor is the population of real, externally published EU member-state Trusted Lists (ETSI TS 119 612), which are selected because they are the rare real, externally maintained, dated status artefacts whose own published `ServiceHistory` of `StatusStartingTime` transitions independently contains the action-time versus review-time structure under study, so the divergence cannot be an artefact of author-chosen data. We take every member-state list reachable from the EU List of Trusted Lists, deposited verbatim and each pinned by SHA-256, with no network access at verification time; the census covers 26 of the 27 member states (Ireland's endpoint was unreachable at fetch) plus the three European Economic Area lists. The Estonian list (retrieved 2026-06-25, SHA-256 `c6b7b872...c435918`) is reported in close-up as the worked example. To remove hand-tuning, each worked example's action time is computed from the file as the midpoint of that service's own last valid window rather than chosen by the author. The unit of analysis is one `TSPService` entry at each list's own granularity, which is not a count of distinct providers, since some lists enumerate per certificate; a transition is counted when an entry's current status is revoked-class and its history records any earlier valid status (`granted`, `under-supervision`, or `accredited`), the full ETSI active vocabulary rather than `granted` alone, so member states using the older statuses are not silently undercounted. The framing is stated honestly: the counts measure the published lists, not real-world incident rates.

The delegated-mandate layer and the dependency closure have no real published artefact — there is no national list of which delegate was authorised for which principal, with what scope, on a given date — so these layers are necessarily studied on controlled synthetic fixtures whose purpose is to vary, on demand, the timing

of a revocation and the loss of a dependency. They are mechanism illustrations and specification scaffolding, not co-equal empirical claims; the certificate and trust-service layer rests on real data, the mandate and delegation layer on synthetic illustration, and the manuscript keeps that distinction explicit throughout. The natural next validation is a real-system instantiation over a live delegation protocol with an executed revocation, named as future work in Section 7.

4.3 The fixed case object

The common object is one synthetic delegated-action evidence episode: synthetic actors and roles, a delegated-authority relationship, a single fixed action transcript, dated status evidence, verifier policies, a verdict vocabulary, and explanation traces. Across experiments only the review context changes — the verification time, the status or trust snapshot consulted, and which dependencies remain available. Three dated reference points anchor the status evidence and are used throughout: $T_0 =$ 2026-05-01 is the action time, at which the delegation is valid; $T_1 =$ 2026-05-08, at which it has been revoked; and $T_2 =$ 2026-05-15, at which the relevant status list is no longer available. The root authority remains valid throughout, isolating drift to the delegation rather than the trust root.

4.4 Solution objectives and design requirements

The problem characterisation of Section 2 and the gaps in Section 3 yield the requirements the designed artefact of Section 6 must meet, which are also the targets the evaluation tests: preserve the action-time authorisation status and the delegation scope; preserve the issuer's dated trust-service state at action time; preserve the dated dependency closure rather than the record alone; bind the record to a timing assumption the verifier can check; and fail closed to a bounded negative verdict rather than to a default acceptance whenever any of these is absent, stale, or contradicted.

4.5 Verifier modes and design rules

Two verifier modes are contrasted. *Current-only lookup* asks whether the relevant authority or trust service is valid now. *Archived dated-status replay* asks whether the relevant status or trust context was acceptable at the action time, by replaying a dated snapshot rather than a live responder. Each harness follows four rules: the action is fixed and only context varies; every verdict is narrow local vocabulary, not an assurance outcome; every positive result is paired with a negative-control boundary — missing, stale, conflicting, policy-incompatible, anchor-missing, or weak-clock, depending on the experiment; and every experiment emits both machine-readable matrices for reproducibility and human-readable explanation traces for legibility to a later reviewer.

4.6 Standards-adjacent cryptographic checks

To keep the case from resting on pure semantic labelling, the harnesses are backed by small local cryptographic adapters over generated fixtures: Ed25519 signing and

verification over canonical JSON evidence payloads, SHA-256 payload hashing, a locally signed timestamp-token simulation bound to the payload hash, and negative controls for a bad signature, an altered payload, and a stale status record. These adapters establish that the dated snapshots the verifier replays are themselves integrity-checked locally; they do not implement, and the paper does not claim, eIDAS trusted-list processing, qualified trust-service validation, live OAuth or GNAP flows, SCITT services, or RFC 3161 timestamp authorities.

4.7 Evaluation strategy

The artefact is evaluated by demonstration plus analytical, argument-based evaluation, not by field deployment, performance benchmarking, or independent replication. Three moves carry the evaluation: (i) demonstration on the real published lists (RQ1) and on the two controlled harnesses (RQ2, RQ3), each positive verdict paired with a falsifying negative control; (ii) the standards-adjacent local cryptographic checks of Section 4.6, which reduce but do not remove the pure-semantic-model character; and (iii) a clause-by-clause security argument that maps each clause of the verification predicate to a stated cryptographic assumption — existential unforgeability under chosen-message attack for the curator signature, collision resistance for the hash, and a sound timestamp anchor — and each security goal to the adversary capability and the negative control that detects its absence (Table 6). What this evaluation is not is stated plainly so the reader is not misled: there is no formal proof of the predicate, no performance benchmark, no field trial, and no third-party replication, and the formal predicate statement and wire encoding are named as future work.

4.8 Scope within the programme and reproducibility

This paper reports the three harnesses that isolate the temporal conflation — status drift across the real published trusted lists, delegation-revocation drift, and dependency closure. Two further single-point-in-time harnesses (receipt-wrapper proof boundaries and freshness) concern a binding-versus-meaning question reported in a concurrent submission by the same author, disclosed in the back matter; they share no experiment, table, or figure with this paper, and the reproducibility package separates the two groups. Each experiment is driven by a recorded script under a single entry point (`scripts/verify-paper.sh`) with a checksum-pinned environment, so the reported matrices regenerate deterministically; the package comprises the three reported experiment directories — including the deposited EU/EEA member-state Trusted Lists and the census script that produces Table 3 — the dependency-closure fixtures, the standards-adapter checks, the negative controls, and a bounded 17-entry source-neighbour ledger across 11 families used only for positioning, and is deposited with a persistent identifier [31]. # 5. Demonstration: experiments and results

The experiments hold one action fixed and move authorisation status through time, demonstrating the problem the artefact of Section 6 is designed to solve: Section 5.1 answers RQ1, Section 5.2 answers RQ2, and Section 5.3 answers RQ3. Section 5.1 establishes the effect on a *real, externally published* trust artefact, so that the

central divergence does not rest on data we authored. Sections 5.2 and 5.3 then isolate two mechanisms a published list cannot vary on demand: the timing of a delegation revocation, and the dependency closure a later verdict needs. The real-list experiment is the empirical anchor and carries the contribution; the two controlled harnesses are mechanism illustrations and specification scaffolding rather than co-equal empirical claims, showing why the effect generalises below the certificate layer and what must be preserved to survive it. Verdict labels throughout are local verifier vocabulary, not assurance, conformance, or legal outcomes.

5.1 Status drift across the EU trusted lists

The first experiment reproduces the action-time/review-time divergence on real, externally published EU member-state Trusted Lists (ETSI TS 119 612), with no synthetic status data. Each national list publishes, for every trust service, a current `ServiceStatus` and a `ServiceHistory` of dated `StatusStartingTime` transitions; the data are real, dated, and externally maintained, and nothing about the status values or their dates is chosen by us. We replay over the Estonian list in close-up — retrieved on 2026-06-25 and deposited verbatim under SHA-256 `c6b7b872…c435918` — and then over every member-state list reachable from the EU List of Trusted Lists (LOTL), so the central divergence is shown not to be a property of one country.

Two verifier modes read this list for a fixed action performed at a fixed action time. *Current-only* lookup reads the service's present `ServiceStatus`, which is what a relying party asking "is this valid now?" against today's list obtains. *Dated replay* resolves the status that obtained at the action time, by selecting the latest transition whose `StatusStartingTime` precedes it, the standard point-in-time resolution a list's own history supports. To avoid any hand-tuning, each worked example's action time is the midpoint of that service's own last valid window, computed from the file.

On the Estonian list, of 66 trust-service entries (each carrying a current `ServiceStatus`), 45 record a transition from a valid status — `granted`, or the older `under-supervision` or `accredited` — to a revoked-class status in their own history, and 44 of those held the valid status for at least thirty days, an operational lifetime rather than a same-day list-migration artefact; 25 of the 45 are specifically `granted`-to-revoked under the current eIDAS vocabulary. The divergence is therefore a population property of the real list, not a single constructed case. Table 2 shows three representative services, all earlier-generation Estonian qualified-certificate services that the supervisory body withdrew on 2017-06-30 as later certificate-authority generations took over. An action performed while such a service was `granted` reads as `withdrawn` under a current-only review today, yet `granted` under dated replay, so a naive later reviewer reaches a different accountability verdict for reasons external to the action. The two controls bound the claim: a service whose status never left `granted` shows no divergence, and an action dated before the service was ever listed returns `not_yet_listed_at_action_time` rather than a false positive.

Table 2: Action-time versus review-time status on the real Estonian Trusted List (66 entries; 45 with a valid-to-revoked transition, 25 of them granted-to-revoked). The three drift rows were withdrawn on 2017-06-30; an action performed when the service was granted is read as withdrawn under a current-only review but granted under dated replay. Status values and dates are read directly from the published list.

Role	Service (real, Estonian TL)	Action time	Current-only	Dated replay
Drift	ESTEID-SK	2016-12-30	`withdrawn`	`granted`
Drift	ESTEID-SK 2007	2016-12-30	`withdrawn`	`granted`
Drift	EID-SK 2007	2016-12-30	`withdrawn`	`granted`
Control	ESTEID2018 (never revoked)	2019-05-04	`granted`	`granted`
Control	action before listing	1992-01-18	`withdrawn`	not yet listed

Estonia is not exceptional. Running the identical replay over every EU member-state Trusted List reachable from the LOTL — 26 of the 27 member states; Ireland's endpoint was unreachable at the time of fetch — the divergence-enabling transition appears in all 26. Across 4,815 trust-service entries, 2,071 (43%) record a valid-to-revoked transition, 1,896 of them over an operational window of at least thirty days (Table 3, Figure 1); the share ranges from 6% on the Danish list, whose published history is sparse, to 81% on the German one, and the three European Economic Area lists add 27 of 71 entries. The unit is one `TSPService` entry at each list's own granularity, which is not a count of distinct providers — some lists enumerate per certificate — so the totals measure the published list, not the provider population. The qualitative result is robust to that caveat: the action-time/review-time conflation a later reviewer faces is structural across the EU trust infrastructure, not a feature of any one national list.

Table 3: Action-time versus review-time status across the EU member-state Trusted Lists (26 of 27; Ireland's endpoint was unreachable at fetch). Column *n* is the count of trust-service (TSPService) entries; *drift* is the number whose own service-status history records a valid-to-revoked transition; the percentage is drift over *n*. Every list shows the effect (EU-26 total: 2,071 of 4,815 entries, 43%). Counts are read directly from each published list and pinned by SHA-256 in the reproducibility package.

TL	n	drift	%	TL	n	drift	%
DE	977	789	81	PT	105	48	46
IT	557	85	15	AT	100	23	23
CZ	547	59	11	NL	97	55	57
ES	424	187	44	LT	86	39	45
FR	344	192	56	EE	66	45	68
HU	337	131	39	HR	44	22	50
SK	192	49	26	LV	40	19	48
RO	167	36	22	LU	34	18	53
PL	137	64	47	FI	19	7	37
BE	135	57	42	DK	16	1	6
EL	133	20	15	SE	14	4	29
BG	115	58	50	MT	10	5	50
SI	111	56	50	CY	8	2	25

The standard already carries the history that makes replay possible; the gap is in how the interface is consulted. A relying party is expected to read the list as published now, so unless the dated `ServiceHistory` is itself replayed against the action time, a list re-issued with a service withdrawn gives a later reviewer no bound snapshot of whether the issuer was eligible, and under what status, when the action was performed.

5.2 Delegation-revocation drift: the layer no list publishes

The real list operates at the certificate and trust-service layer. The layer that delegated-authority accountability turns on is the delegate's mandate and scope, and no published trusted list carries it: there is no national list of which delegate was authorised to act for which principal, with what scope, on a given date. The second experiment therefore moves the same contrast one layer up, into a controlled synthetic harness, precisely because the real-world artefact for delegation status does not yet exist. It carries a synthetic delegation chain, the fixed action transcript, dated status records at T_0, T_1, and T_2, the two verifier modes, and negative controls for a missing trusted clock and a conflicting archive. At T_0 the delegation is valid, at T_1 it is revoked, and at T_2 its status list is gone, while the root authority stays valid throughout.

The two modes diverge on the same action, and only one is stable (Table 4, Figure 2).

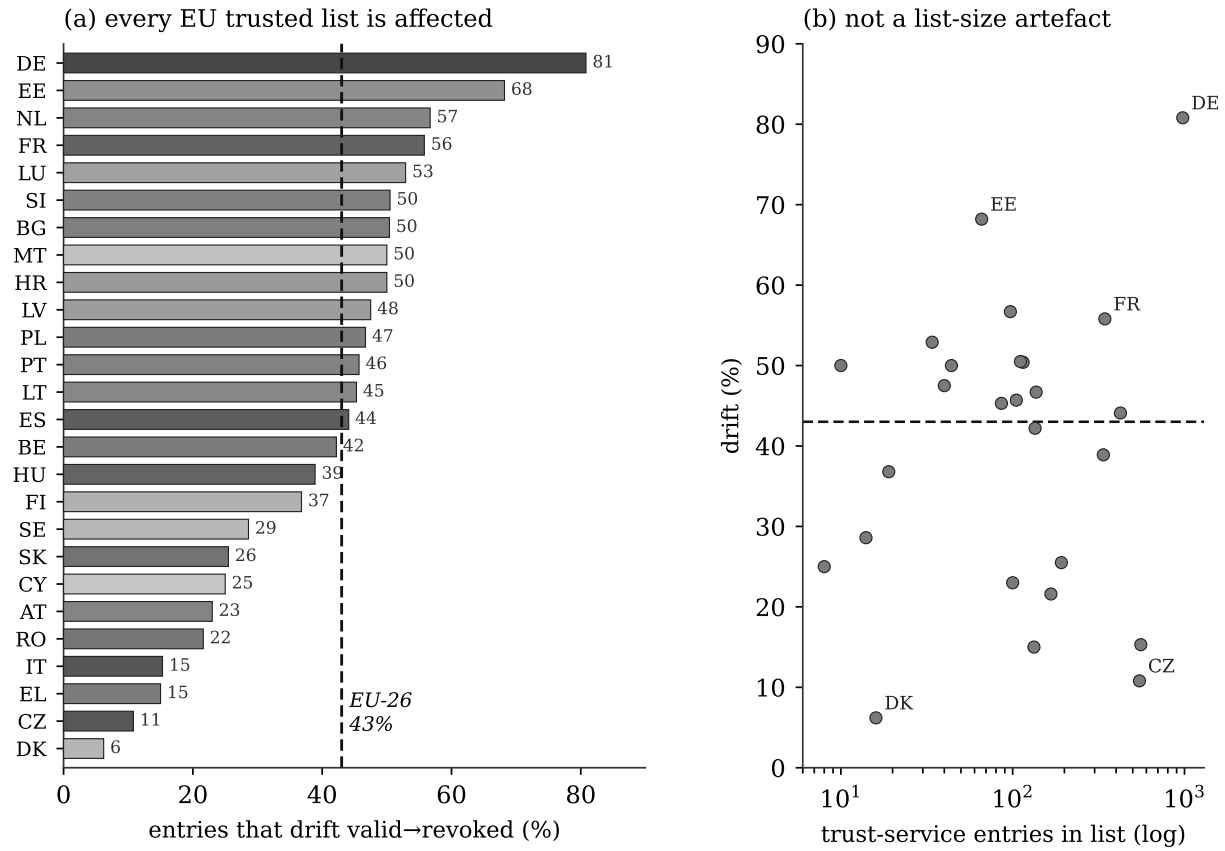

Figure 1: The EU member-state trusted-list census. (a) The share of entries that drift from a valid to a revoked status, by member state, ordered high to low and shaded by list size; every one of the 26 lists is affected, and the dashed line marks the EU-wide 43%. (b) Drift share against list size on a logarithmic scale: the effect is roughly independent of how many entries a list carries, so it is not an artefact of large lists. Greyscale-safe; the same data appear by count in Table 3.

Current-only lookup tracks the present: it returns `valid_currently` at T_0, `revoked_at_review` once the delegation is revoked at T_1, and `stale_or_missing_status` once the status list is unavailable at T_2. Dated replay returns `valid_when_performed` at all three review times, because it answers a question fixed to action time rather than to review time.

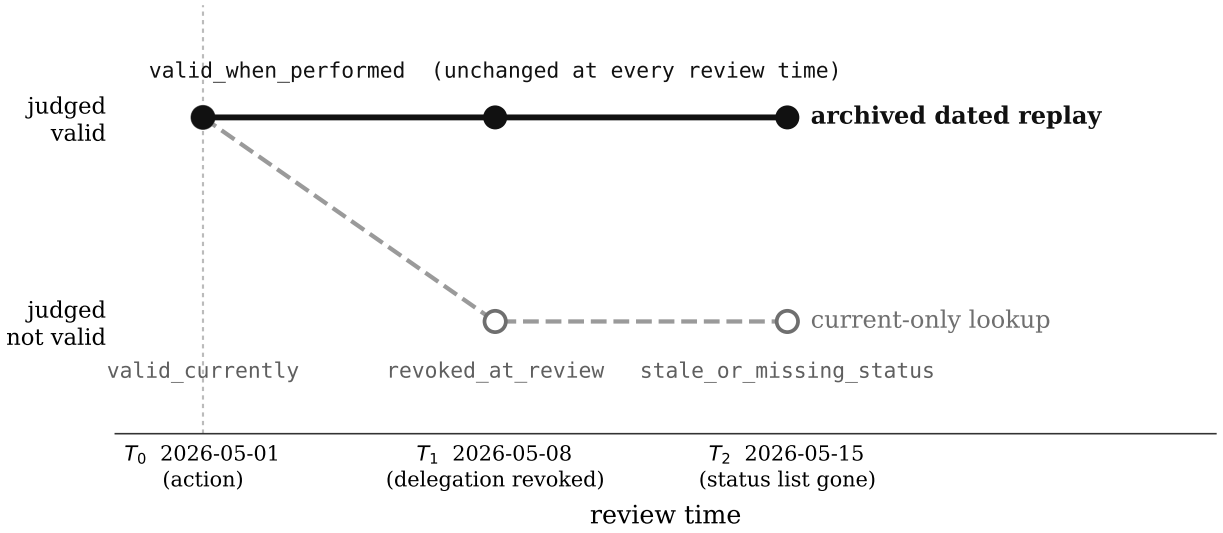

Figure 2: Delegation-revocation drift on one fixed action across three review times (T_0 = 2026-05-01, T_1 = 2026-05-08, T_2 = 2026-05-15). The archived dated-replay track (solid, filled markers) holds valid-when-performed at every review time, while the current-only track (dashed, hollow markers) collapses as the world moves on, from valid-currently to revoked-at-review to stale-or-missing-status. Line style and marker fill, not colour, encode the mode, so the contrast survives greyscale.

Table 4: Delegation-revocation drift for one fixed action under two verifier modes at three review times ($T_0 = $ 2026-05-01, action time; $T_1 = $ 2026-05-08, delegation revoked; $T_2 = $ 2026-05-15, status list gone). Current-only review changes verdict as the world moves on, while dated replay stays fixed to action time. Verdict labels are local verifier vocabulary.

Review time	Current-only lookup	Archived dated-status replay
T_0 — 2026-05-01 (action time)	`valid_currently`	`valid_when_performed`
T_1 — 2026-05-08 (delegation revoked)	`revoked_at_review`	`valid_when_performed`
T_2 — 2026-05-15 (status list gone)	`stale_or_missing_status`	`valid_when_performed`

The negative controls bound the claim. When the action transcript lacks trusted-clock material, replay cannot bind the action to a time and returns `unverifiable_timing` rather than a false positive; when the archive disagrees with the action-time record, replay returns `conflicting_archived_status`. Dated replay therefore preserves the

valid-when-performed distinction that current-only review collapses, but only when the archive is available, internally coherent, and bound to a timing assumption the verifier accepts. Where it cannot, it says so rather than defaulting to acceptance. The same divergence thus appears on a real list at the certificate layer and in the harness at the delegation layer, and the layer that most needs it is the one no published list yet records.

5.3 Dependency closure

The third experiment treats the evidence package as a dependency graph and asks which records must survive together for a bounded verdict to remain reproducible. A complete closure for the modelled episode preserves nine dependency classes: the primary evidence file, schema, policy, issuer key metadata, action-time status record, issuer metadata, receipt or archive pointer, carrier context, and explanation trace. The harness emits 16 closure scenarios, of which two are controls, that remove, stale, or contradict these classes one at a time.

Removing any single supporting class collapses the closure (Table 5). Removing the schema, policy, key metadata, status record, issuer metadata, receipt pointer, or explanation trace yields `missing_dependency`, and removing the carrier context yields `unsupported_conclusion`; in each case the preserved count falls from nine to eight. Staling the status or key metadata yields `stale_dependency`, a contradictory policy or issuer record yields `contradictory_dependency`, and removing the explanation trace under a partial-closure policy yields `partial_closure`. The two controls fix the boundaries of the claim. Removing an artefact outside the dependency graph leaves the closure complete, while preserving only the primary evidence file, one class of nine, yields `missing_dependency` with everything that explains the action gone.

Table 5: Dependency closure over the modelled episode (16 scenarios, two of them controls). The primary-only control shows that keeping the signed record while losing its context leaves the verdict unreproducible.

Scenario class	Count	Outcome
Baseline (all 9 classes preserved)	1	`complete_closure`
Single class removed	8	`missing_dependency` / `unsupported_conclusion`
Class staled (status, key metadata)	2	`stale_dependency`
Class contradicted (policy, issuer)	2	`contradictory_dependency`
Partial-closure policy (explanation trace)	1	`partial_closure`
Control: unrelated artefact removed	1	`complete_closure`

Scenario class	Count	Outcome
Control: primary evidence file only	1	`missing_dependency` (1 of 9 preserved)

Dependency closure converts the temporal results of 5.1 and 5.2 into a preservation requirement. The dated status and trust snapshots that distinguish action time from review time are themselves dependencies; preserving the action record without them, or with them stale or contradictory, returns the reviewer to a current-only position by another route. The unit a later reviewer needs is the dated closure that surrounds the signed file at its centre. # 6. The mandate-status statement: artefact design and security evaluation

This section answers RQ4. It designs the evidence object the preceding experiments require and evaluates it against the design objectives of Section 4.4 — by demonstration, by the negative controls of Section 5, and by an analytical security argument that maps each clause of its verification predicate to a stated cryptographic assumption (Table 6). The three experiments converge on a single requirement. For a later accountability verdict over a delegated action to be reproducible, the evidence interface must preserve, as a dated object bound to the action and not as a set of live queries, three things: the action-time authorisation status, including the delegation grant and its scope; the action-time state of the trust service or list behind the issuer; and the dependency closure that lets a verifier explain the bounded verdict. Each is something the experiments show is lost when consulted current-only or preserved without its context. The requirement is deliberately narrow: it says nothing about whether the action was wise, lawful, or safe, only about whether the verdict that an authority was in force at T can be reproduced at $T + k$ by someone other than the original relying party. Long-term-validation procedures already give retrospective certificate validity [17], and evidence-record syntaxes already give a durable container for preserved data [18, 19]; what neither mandates is that the preserved, dated object be the action-time mandate status rather than a signature or a file.

We make the object concrete as a minimal specification, not a full protocol. A *mandate-status statement* is a signed record carrying: a reference binding it to the action transcript; the action time, anchored by a proof of existence; the delegation grant and its scope with the validity window claimed; the issuer's dated trusted-list or trust-service state; and a manifest of the dependency closure, listed by hash. A later verifier returns `valid_when_performed` only when every clause of a verification predicate holds: the curator signature verifies under an accepted anchor; the proof of existence places the action time before the statement was anchored; the validity window contains the action time and the dated mandate status is not revoked at that time; the issuer's dated trust state is `granted`; and the closure manifest resolves completely. A failed clause yields the corresponding bounded negative verdict from Section 5, never a default acceptance. Figure 3 shows the predicate as a chain of five clauses, each guarding the bounded negative verdict it exits to.

The security argument is what distinguishes the statement from simply wrapping a record, and each clause rests on a stated cryptographic assumption rather

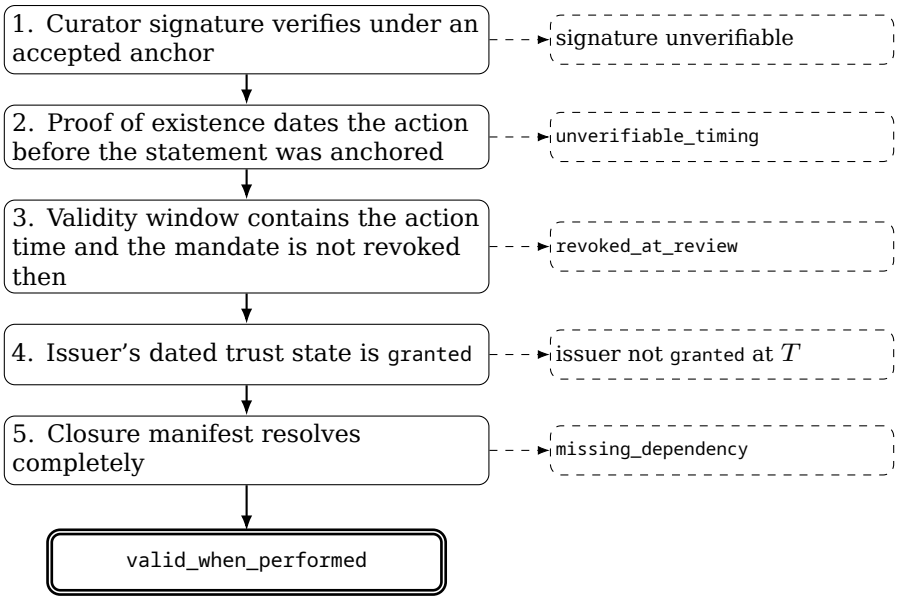

Figure 3: The mandate-status statement's verification predicate. All five clauses must hold for a verifier to return `valid_when_performed` (double border); a failed clause exits to the bounded negative verdict it guards (dashed), never to a default acceptance. Border style, not colour, encodes pass versus fail, so the figure survives greyscale.

than on prose alone. Unforgeability follows from the curator signature under the existential-unforgeability of that signature scheme against chosen-message attack: an adversary cannot forge a `valid_when_performed` verdict without the anchoring key, which is the trust the threat model relocates to a named curator rather than removing. Timing-binding integrity follows from the proof of existence on the action time, assuming a sound timestamp anchor and a collision-resistant hash binding the action: the asserted action time cannot be dated later than the anchor and is bound by hash to the status it is checked against. The proof of existence is an upper bound, not a witness of the true action instant, so a curator who asserts an action time inside a `granted` window is not caught by it; lower-bounding the true instant needs an independent action-time witness, and absent one that case is the honesty-at-capture residual conceded in Section 7, not a timing failure. Closure-completeness follows from the hash manifest under the same collision-resistance assumption, so a missing, staled, or contradicted dependency is detected rather than silently tolerated, relative to the manifest's own enumeration: a manifest that under-lists a true dependency class still yields an internally consistent verdict over an incomplete closure, detectable only against an external schema of required classes and otherwise part of the same conceded residual. These are the standard assumptions of the signature, hash, and timestamping primitives the statement composes; the contribution is the predicate that binds them to action-time mandate status, not new cryptography. An evidence-record syntax is a suitable *container* for such a statement — it renews the integrity of whatever bytes it wraps [18] — but it does not define that the wrapped object is the action-time mandate status, nor the verification predicate that separates valid-when-performed from revoked-at-review. The statement also composes with a transparency service for auditable existence

[14] and a qualified timestamp for its date [8].

The hardest case is the producer who also curates the archive: such an insider can sign a faithful container around a substantively wrong snapshot, and for that adversary the statement reduces to an evidence-record container, the residual conceded in Section 7. The structured dated object still exposes a cross-check the bare container does not. Because a published trusted list carries the dated service-status history that Section 5.1 replays, an independent monitor can compare the issuer trust state asserted in a statement against the public list's own history at the action time, and registering the statement in a transparency log at capture makes a later back-dating detectable. This narrows the honest-at-capture residual to the part not covered by any externally published anchor, without claiming to remove it. That cross-check reaches only the issuer's trusted-list state; the delegation grant, its scope, and the asserted action time — the mandate layer no published list records, and the article's central contribution — have no external anchor today, so their residual is reducible only by registering the statement at capture, by qualified-preservation supervision, or by a future mandate registry, not by the public-list comparison. We specify the object and its checks at the level the experiments falsify; the full wire encoding and a formal statement of the predicate are future work, not a finished protocol.

Table 6 states the security goals the predicate is meant to meet, the adversary capability each addresses, the negative control that exercises it, and the verdict returned when the goal is not met. The mapping is the security reading of the negative controls already run in Section 5: each control is the point at which a specific guarantee, if absent, is detected rather than assumed.

Table 6: Security goals, the adversary capability each addresses, the negative control that exercises it, and the bounded verdict returned when the goal is not met. The final row is the residual the threat model concedes rather than claims to remove.

Security goal	Adversary capability	Negative control	Verdict when unmet
Timing-binding integrity	Back- or post-date the action vs status	Action lacks trusted-clock material	`unverifiable_timing`
Curator-equivocation detection	Sign a snapshot conflicting with the record	Archive disagrees with action-time record	`conflicting_archived_status`
Freshness of status	Replay a stale or absent status	Status list staled or missing	`stale_or_missing_status`
Closure completeness	Drop or contradict a dependency	Dependency removed, staled, or contradicted	`missing_dependency` / `contradictory_dependency`

Security goal	Adversary capability	Negative control	Verdict when unmet
Coverage boundary	Claim status before the service was listed	Action dated before listing (real list)	`not_yet_listed_at_action_time`
Honesty at capture	Sign a faithful but substantively wrong snapshot	Out of scope — named, accountable curator (§7)	not bounded; declared residual

As a deployment consequence, evidence infrastructure should make claim boundaries explicit and dated: a record that supports only valid-under-dated-replay must not be silently restated as authorised, compliant, or safe, and a verdict available only from a current-time lookup should be marked review-time-dependent and not reproducible. The distinction between current-time and action-time status should travel with the evidence rather than living in the memory of whoever verified it first.

7. Limitations and threats to validity

These limitations are the boundary conditions of an internal-validity design evaluation (Section 4.1): they bound how far the demonstrated behaviour generalises beyond the one real list and the synthetic harnesses, not whether the mechanism behaves as specified, which the experiments and their negative controls settle.

The archive-trust residual. The central limitation is the one the threat model names: archived action-time replay relocates trust rather than removing it. The replayed snapshot is only as good as the party who curated and signed it and the timestamp that anchors its date; as Section 2 sets out, the scheme guarantees the snapshot's integrity but not the truthfulness of what was captured, and even the timestamping anchor may retrospectively prove untrustworthy [9, 18]. We state this as a security-considerations item, not a refutation, and bound it with the field's own mechanisms. Every verification scheme bottoms out in at least one accepted trust anchor [6]; demanding a zero-anchor preservation scheme demands what no cryptographic system provides, and the live alternative, re-querying a current responder, is strictly weaker, because it trusts a present party to reconstruct the past with no dated artefact to check. Archived replay is thus a trust reduction, freezing a live dependency into a dated, checkable one. The residual is bounded and renewable rather than infinite: evidence-record renewal re-anchors to currently strong cryptography before algorithms weaken [18], and keyless, server-side time-stamping reduces the time anchor's dependence on any single long-lived key [11]; qualified preservation services bind the curator to an audited, supervised, liable provider [4, 32]; and append-only transparency makes a misbehaving anchor's behaviour publicly falsifiable by independent monitors, given a gossip or consistency-audit layer that defeats split-view equivocation [12, 33]. The field's accepted criterion of success is detectable, accountable, renewable trust, not trustlessness, and the paper claims no more than that.

Synthetic, single-case, self-authored verdicts. The case object is local, synthetic, and intentionally small, which makes the evidence trail inspectable but limits generalisation. The verdict labels are vocabulary the author also authored, so the experiments establish internal consistency — the verifier returns the specified label under the specified dated context, with negative controls — rather than an external truth oracle. The local cryptographic adapters reduce, but do not remove, the pure-semantic-model character of the study, and the bounded source-neighbour search records no exact match without claiming absolute novelty. The appropriate claim is a reproducible synthetic case study of action-time versus review-time authorisation drift with cautious positioning against neighbouring standards.

Standards adjacency is not conformance. The harnesses model verifier semantics and implement only small local adapters. They do not implement eIDAS trusted-list processing, qualified trust-service validation, live OAuth or GNAP flows, SCITT services, certificate-transparency logs, or RFC 3161 timestamp authorities, and the paper makes no conformance, certification, legal-effect, production-readiness, or third-party-validation claim. Future work should test whether the preservation predicate survives larger multi-agent episodes, real organisational evidence packages, longer retention horizons, and independent reviewer replication.

8. Conclusion

The article asked whether a later reviewer can reproduce the action-time accountability verdict over a delegated action once authorisation status has drifted, what an evidence interface must preserve for that to be possible, and what designed object satisfies that requirement. The answers close the questions in turn. To the overarching question and RQ1: a current-only interface cannot reproduce the verdict, because its answer is dated to review time, and the divergence is a property of real published data — across the EU member-state Trusted Lists an action performed while a qualified service was valid reads as `withdrawn` under a current-only review yet valid under dated replay, and 2,071 of 4,815 service entries across 26 of the 27 lists carry such a transition — every list affected, 45 of 66 on the Estonian list. To RQ2: the same split appears one layer up, at the delegated mandate and scope that no published list records, where current-only review moves through `valid_currently`, `revoked_at_review`, and `stale_or_missing_status` while dated replay holds `valid_when_performed`, and the negative controls return `unverifiable_timing` or `conflicting_archived_status` rather than false positives. To RQ3: the unit a later reviewer must preserve is not the signed record but the dated dependency closure around it, since removing, staling, or contradicting any one supporting class collapses the bounded verdict, and the primary-evidence-only control yields `missing_dependency`. To RQ4: a dated, replayed mandate-status statement, gated by a five-clause verification predicate over stated cryptographic assumptions, can reproduce the action-time verdict, provided it preserves the action-time authorisation status, the issuer's dated trust state, and the dependency closure together. Existing standards make cryptographic validity, the existence of data, and the integrity of log entries survive over time, but reason at the level of keys, certificates, and log entries; the layer that delegated-authority review turns on is the delegate's mandate and delegation status, preserved and replayed

as dated evidence. Preserving it does not remove trust but relocates it to a named, accountable, renewable anchor — the detectable-and-renewable standard of success the field already accepts.

Declaration of competing interest

The author developed and maintains the harnesses, fixtures, and verifier vocabulary evaluated here, so this is an artefact-based laboratory study by the tool's own author rather than an independent third-party evaluation; the negative controls, the deposited reproducibility package, and the explicit treatment of residual trust are provided so that the verdicts can be checked by others. The author is separately employed in the digital-trust and electronic-identity sector in a role that does not involve the subject matter of this article; no employer systems, data, or endorsement are involved, and the eIDAS and PKI material is used only as an analytic reference, with no product or service implied. There is no financial competing interest to declare.

Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

CRediT author statement

Anton Sokolov: Conceptualization, Methodology, Software, Validation, Investigation, Data curation, Writing — original draft, Writing — review and editing, Visualization, Project administration.

Data availability

The delegation and dependency-closure experiments run over local synthetic fixtures; the trusted-list experiment of Section 5.1 runs over the real EU member-state Trusted Lists reachable from the EU LOTL (ETSI TS 119 612) — 26 of the 27 member states plus the three EEA lists, with the Estonian list reported in detail — each deposited verbatim and pinned by SHA-256, with no network access at verification time. The reproducibility package for this paper is the delegation-revocation directory (`experiments/01-delegation-revocation`), the real trusted-list directory (`experiments/08-real-eu-trusted-list`), and the dependency-closure directory (`experiments/04-dependency-closure`), with their fixtures, negative controls, standards-adapter checks, and a bounded source-neighbour search ledger; it regenerates from a single recorded entry point (`scripts/verify-paper.sh`) and is openly deposited on Zenodo with a persistent identifier [31] that links to the public project repository, under CC BY 4.0 for data and the MIT licence for code. The two single-point-in-time harnesses that belong to the concurrent submission identified below, and an earlier synthetic trusted-list prototype superseded by the real-list experiment, are not part of this deposit. The

study uses only synthetic fixtures and publicly published trusted lists; it involves no human subjects, no personal data, and no animal experimentation.

Related submissions and prior dissemination

A sibling manuscript by the author, *Freshness Is Not Authenticity: Cross-Verifier Failure Modes and a Layered Verdict Model for AI-Agent Evidence Packages* (submitted to the Journal of Cybersecurity, manuscript ID CyberSec-2026-254, 2026-06-24), studies a different question on the same broad theme. Its threat model is cross-verifier disagreement over a single freshly produced evidence package at one point in time, and its result is a layered verdict model; the present paper's threat model is a single fixed action reviewed after status has drifted, and its result is that action-time and review-time status are two dated facts. The present paper is restricted to that temporal axis (revocation and trusted-list drift, and dependency closure), which the sibling does not test; the receipt-wrapper and freshness harnesses that fall in the sibling's scope are excluded here. The two manuscripts share no experiment, table, or figure. A further companion note by the author, the eIDAS reference-mapping survey cited here only as background [30], is a separate manuscript of a different genre that was submitted to another journal and declined; it is named here so the relationship is on the reviewer-visible record, not only in the cover letter. An earlier development preprint of the wider laboratory programme, reporting all of the harnesses together, was prepared before this split; the present paper consolidates and supersedes its temporal-spine material, and the receipt and freshness material is reported in the sibling. This relationship is disclosed for transparency and restated in the cover letter.

Declaration of generative AI in the writing process

During the preparation of this work the author used a generative-AI assistant (Anthropic Claude Code) for drafting, formatting, and reproducibility checks. The author reviewed and edited all content, verified every reported figure against the underlying run artefacts, and takes full responsibility for the content and the decision to submit. The assistant was not used to create or alter any data, results, or images, and is not credited as an author.

References

1. W3C. Verifiable Credentials Bitstring Status List v1.0. 2025. https://www.w3.org/TR/vc-bitstring-status-list/.
2. Richer J (Ed). RFC 7662: OAuth 2.0 Token Introspection. IETF, 2015. https://www.rfc-editor.org/rfc/rfc7662.
3. IETF GNAP Working Group. Grant Negotiation and Authorization Protocol (GNAP) Core Protocol. RFC 9635, 2024. https://www.rfc-editor.org/rfc/rfc9635.

4. European Parliament and Council. Regulation (EU) No 910/2014 on electronic identification and trust services (eIDAS). 2014. https://eur-lex.europa.eu/eli/reg/2014/910/oj.
5. ETSI. TS 119 612: Electronic Signatures and Infrastructures (ESI); Trusted Lists. https://www.etsi.org/standards.
6. Reddy R, Wallace C. RFC 6024: Trust Anchor Management Requirements. IETF, 2010. https://www.rfc-editor.org/rfc/rfc6024.
7. IETF OAuth Working Group. Token Status List. Internet-Draft draft-ietf-oauth-status-list, 2025. https://datatracker.ietf.org/doc/draft-ietf-oauth-status-list/.
8. Adams C, Cain P, Pinkas D, Zuccherato R. RFC 3161: Internet X.509 PKI Time-Stamp Protocol (TSP). IETF, 2001. https://www.rfc-editor.org/rfc/rfc3161.
9. Pinkas D, Pope N, Ross J. RFC 3628: Policy Requirements for Time-Stamping Authorities (TSAs). IETF, 2003. https://www.rfc-editor.org/rfc/rfc3628.
10. Haber S, Stornetta WS. How to Time-Stamp a Digital Document. Journal of Cryptology, 3(2):99-111, 1991. https://doi.org/10.1007/BF00196791.
11. Buldas A, Kroonmaa A, Laanoja R. Keyless Signatures' Infrastructure: How to Build Global Distributed Hash-Trees. Proc. NordSec 2013, LNCS 8208, pp. 313-320. https://doi.org/10.1007/978-3-642-41488-6.
12. Laurie B, Langley A, Kasper E. RFC 6962: Certificate Transparency. IETF, 2013. https://www.rfc-editor.org/rfc/rfc6962.
13. Laurie B, Messeri E, Stradling R. RFC 9162: Certificate Transparency Version 2.0. IETF, 2021. https://www.rfc-editor.org/rfc/rfc9162.
14. IETF SCITT Working Group. An Architecture for Supply Chain Integrity, Transparency, and Trust (SCITT). Internet-Draft draft-ietf-scitt-architecture-12, 2025. https://datatracker.ietf.org/doc/draft-ietf-scitt-architecture/.
15. Newman Z, Meyers JS, Torres-Arias S. Sigstore: Software Signing for Everybody. Proc. ACM CCS 2022. https://doi.org/10.1145/3548606.3560596.
16. Coalition for Content Provenance and Authenticity. C2PA Technical Specification. https://c2pa.org/specifications/.
17. ETSI. EN 319 102-1: Procedures for Creation and Validation of AdES Digital Signatures; Part 1: Creation and Validation. (POE, best-signature-time, B-LT/B-LTA long-term validation levels.) https://www.etsi.org/standards.
18. Gondrom T, Brandner R, Pordesch U. RFC 4998: Evidence Record Syntax (ERS). IETF, 2007. https://www.rfc-editor.org/rfc/rfc4998.
19. Jerman Blazic A, Saljic S, Gondrom T. RFC 6283: Extensible Markup Language Evidence Record Syntax (XMLERS). IETF, 2011. https://www.rfc-editor.org/rfc/rfc6283.
20. National Institute of Standards and Technology. Artificial Intelligence Risk Management Framework (AI RMF 1.0). NIST AI 100-1, 2023. https://doi.org/10.6028/NIST.AI.1.

21. Schneier B, Kelsey J. Cryptographic Support for Secure Logs on Untrusted Machines. Proc. 7th USENIX Security Symposium, 1998, pp. 53-62. https://www.usenix.org/legacy/publications/library/proceedings/sec98/.

22. Bellare M, Yee B. Forward Integrity for Secure Audit Logs. Technical Report CS98-580, Dept. of Computer Science and Engineering, University of California San Diego, 1997.

23. Crosby SA, Wallach DS. Efficient Data Structures for Tamper-Evident Logging. Proc. 18th USENIX Security Symposium, 2009, pp. 317-334.

24. Ma D, Tsudik G. A New Approach to Secure Logging. ACM Transactions on Storage, 5(1):2:1-2:21, 2009. https://doi.org/10.1145/1502777.1502779.

25. South T, Marro S, Hardjono T, Mahari R, Whitney CD, Greenwood D, Chan A, Pentland A. Authenticated Delegation and Authorized AI Agents. arXiv:2501.09674, 2025. https://arxiv.org/abs/2501.09674.

26. Jones M, Nadalin A, Campbell B (Ed), Bradley J, Mortimore C. RFC 8693: OAuth 2.0 Token Exchange. IETF, 2020. https://www.rfc-editor.org/rfc/rfc8693.

27. OWASP GenAI Security Project. Agentic AI: Threats and Mitigations, v1.0. 2025. https://genai.owasp.org/resource/agentic-ai-threats-and-mitigations/.

28. Cloud Security Alliance. Agentic AI Identity and Access Management: A New Approach. 2025. https://cloudsecurityalliance.org/artifacts/agentic-ai-identity-and-access-management-a-new-approach.

29. Sokolov A. Cryptographic Attestation for AI Agent Governance under the EU AI Act: A Survey of Approaches and Standards. Zenodo, 2026. https://doi.org/10.5281/zenodo.20357731.

30. Sokolov A. Operationalizing the EU AI Act through eIDAS Trust Services Primitives: A Standards-Gap Reference Mapping. Zenodo, 2026. https://doi.org/10.5281/zeno

31. [dataset] Sokolov A. The Time-Travel Evidence Lab: temporal-spine reproducibility package (delegation-revocation drift, real EU trusted-list status replay, dependency closure). Zenodo, 2026. https://doi.org/10.5281/zenodo.20840168.

32. ETSI. TS 119 511: Policy and security requirements for trust service providers providing long-term preservation of digital signatures or general data. https://www.etsi.org/standards.

33. Yu J, Cremers C, Ryan MD. DTKI: A New Formalized PKI with Verifiable Trusted Parties. arXiv:1408.1023, 2014. https://arxiv.org/abs/1408.1023.